

The Neutrino Packages

dist · poly · finance · astro · rmt — with machine-verified examples

Mico Mrkaic

July 2026

Contents

A package is a file of let definitions; `load("packages/name.nu")` runs it in the current session and its bindings persist. Records of closures act as namespaces (`norm.cdf`), and packages validate their inputs with `assert`, so they fail like builtins do — with a message, not an accident. Every transcript below is machine-verified against the interpreter by `tests/run_manual.sh`, and every numerical claim is cross-checked in the package's own golden tests against an independent reference (SciPy, NumPy, Python's `datetime`, or the `astral` library — named in each section).

1. dist.nu — probability distributions

Six distributions — `norm`, `student`, `chi2`, `fdist`, `expo`, `unif` — each a record with `pdf`, `cdf`, `inv` (quantile), and `rand`. Parameters are explicit: `norm.cdf(x, mu, sigma)`. Quantiles with no closed form use bracket expansion plus `fzero` on the CDF; the quantile domain is $0 < p < 1$. Cross-checked against SciPy (`tests/34_dist.test`).

```
neutrino> load("packages/dist.nu")
neutrino> norm.cdf(1.96, 0, 1)
0.975002
neutrino> norm.inv(0.975, 0, 1)
1.95996
neutrino> student.inv(0.975, 30)
2.04227
neutrino> chi2.inv(0.95, 3)
7.81473
neutrino> fdist.cdf(3.0, 4, 20)
0.956799
```

A complete two-sample t-test — simulate, test, decide:

```
neutrino> load("packages/dist.nu")
neutrino> rng(42)
neutrino> let x1 = norm.rand(40, 100, 15)
[75.8016; 123.017; 111.725; 93.9971; 100.238; 98.0904; 107.158; 90.1486; 90.4082; 94.4609; 96.6851;
neutrino> let x2 = norm.rand(40, 108, 15)
[102.897; 92.0654; 112.122; 98.9768; 112.891; 112.931; 92.8931; 102.642; 111.903; 100.646; 100.94;
neutrino> let t = (mean(x2) - mean(x1)) / sqrt(var(x1)/40 + var(x2)/40)
2.46819
neutrino> 2 * (1 - student.cdf(abs(t), 78))
```

0.0157683

Out-of-domain probabilities refuse with the package's own message:

```
neutrino> load("packages/dist.nu")
error: quantile: p must be in (0,1), got 1.5
neutrino> student.inv(1.5, 10)
```

Function	Worked example	Result
norm.pdf/cdf/inv(x, mu, sigma)	norm.cdf(1.96, 0, 1)	0.975002
norm.rand(n, mu, sigma)	mean(norm.rand(500, 10, 2)) > 9	true
student.pdf/cdf/inv(x, v)	student.inv(0.975, 30)	2.04227
chi2.pdf/cdf/inv(x, k)	chi2.inv(0.95, 3)	7.81473
fdist.pdf/cdf/inv(x, d1, d2)	fdist.cdf(3.0, 4, 20)	0.956799
expo.pdf/cdf/inv(x, rate)	expo.inv(0.5, 2)	0.346574
unif.pdf/cdf/inv(x, a, b)	unif.cdf(3, 0, 10)	0.300000

2. poly.nu — polynomials

Coefficients are row vectors, highest power first: $[2, -3, 1]$ is $2x^2 - 3x + 1$. Roots come from the companion matrix and eig — the same algorithm Octave uses, on Neutrino's LAPACK-verified eigensolver. Cross-checked against NumPy (tests/37_poly.test).

```
neutrino> load("packages/poly.nu")
neutrino> polyval([2, -3, 1], 4)
21
neutrino> roots([1, -6, 11, -6])'
[1, 2, 3]
neutrino> roots([1, 0, 1])'
[1i, -1i]
neutrino> companion([1, -6, 11, -6])
[6, -11, 6; 1, 0, 0; 0, 1, 0]
```

Least-squares fitting (Vandermonde + backslash), and calculus that inverts:

```
neutrino> load("packages/poly.nu")
neutrino> let x = [0, 1, 2, 3, 4]
[0, 1, 2, 3, 4]
neutrino> let y = [1.1, 2.9, 9.2, 19.1, 32.8]
[1.1, 2.9, 9.2, 19.1, 32.8]
neutrino> polyfit(x, y, 2)
[1.95714, 0.131429, 1.01429]
neutrino> polyder([1, -6, 11, -6])
[3, -12, 11]
neutrino> polyint(polyder([1, -6, 11, -6]), -6)
[1, -6, 11, -6]
neutrino> conv([1, -1], [1, -2])
[1, -3, 2]
```

Function	Worked example	Result
polyval(c, x)	polyval([2, -3, 1], 4)	21
roots(c)	sort(real(roots([1, -6, 11, -6])))'	[1.00000, 2.00000, 3.00000]
companion(c)	companion([1, 0, -4])	[0.00000, 4.00000; 1.00000, 0.00000]
polyfit(x, y, n)	polyfit([1, 2, 3], [2, 5, 10], 2)	[1.00000, -8.32667e-16, 1.00000]
polyder(c)	polyder([1, -6, 11, -6])	[3, -12, 11]
polyint(c, k)	polyint([3, -12, 11], -6)	[1.00000, -6.00000, 11.0000, -6.00000]
conv(a, b)	conv([1, -1], [1, -2])	[1.00000, -3.00000, 2.00000]

3. finance.nu — the HP-12C's greatest hits

Time value of money, cash flows, bonds, amortization, and date arithmetic. Sign convention (HP-12C): money received is positive, money paid is negative; rates are per period, as decimals. Cross-checked against SciPy and Python's datetime (tests/38_finance.test).

The mortgage block — payment, implied rate, savings growth:

```
neutrino> load("packages/finance.nu")
neutrino> pmt(360, 0.005, 250000, 0)
-1498.88
neutrino> rate(360, 250000, -1498.876313, 0)
0.005
neutrino> fv(120, 0.004, 0, -200)
30726.4
```

Cash flows and bonds:

```
neutrino> load("packages/finance.nu")
neutrino> npv(0.10, [-1000, 300, 420, 680])
130.729
neutrino> irr([-1000, 300, 420, 680])
0.163406
neutrino> bond_price(100, 0.08, 0.06, 10, 1)
114.72
neutrino> bond_ytm(114.7202, 100, 0.08, 10, 1)
0.06
neutrino> bond_mduration(100, 0.08, 0.06, 10, 1)
7.0236
```

amort returns the whole schedule as a record of columns — a small data frame you can index, sum, and plot:

```
neutrino> load("packages/finance.nu")
neutrino> let s = amort(1000, 0.01, 3)
{period = [1; 2; 3], payment = [340.022; 340.022; 340.022], interest = [10; 6.69978; 3.36656], principal = [12]}
neutrino> fields(s)'
["period", "payment", "interest", "principal", "balance"]
neutrino> s.payment[1]
```

```

340.022
neutrino> s.balance'
[669.978, 336.656, 4.26326e-12]
neutrino> sum(s.principal)
1000

```

Dates, HP-12C style — actual day counts, date arithmetic, day of week, and the bond market's 30/360 convention. `datestr` returns a display string, `daterec` the {y, m, d} record, and `today()` a serial day number, so arithmetic reads naturally — `datestr(today() + 90)` is the date in 90 days:

```

neutrino> load("packages/finance.nu")
neutrino> days(2026, 1, 1, 2026, 7, 9)
189
neutrino> datestr(datenum(2026, 7, 17) + 90)
"2026-10-15"
neutrino> daterec(datenum(2024, 2, 29))
{y = 2024, m = 2, d = 29}
neutrino> dateadd(2024, 2, 28, 2)
{y = 2024, m = 3, d = 1}
neutrino> dow(2026, 7, 9)
4
neutrino> days360(2024, 1, 31, 2024, 7, 31)
180
neutrino> daterec(today()).y >= 2026
true

```

Function	Worked example	Result
<code>pmt(n, i, pv, fv)</code>	<code>pmt(360, 0.005, 250000, 0)</code>	-1498.88
<code>pv(n, i, pmt, fv)</code>	<code>pv(360, 0.005, -1498.88, 0)</code>	250001.
<code>fv(n, i, pv, pmt)</code>	<code>fv(120, 0.004, 0, -200)</code>	30726.4
<code>nper(i, pv, pmt, fv)</code>	<code>nper(0.005, 250000, -1498.88, 0)</code>	359.998
<code>rate(n, pv, pmt, fv)</code>	<code>rate(360, 250000, -1498.876313, 0)</code>	0.00500000
<code>npv(r, cfs)</code>	<code>npv(0.10, [-1000, 300, 420, 680])</code>	130.729
<code>irr(cfs)</code>	<code>irr([-1000, 300, 420, 680])</code>	0.163406
<code>bond_price(face, crate, y, n, freq)</code>	<code>bond_price(100, 0.08, 0.06, 10, 1)</code>	114.720
<code>bond_ytm(price, face, crate, n, freq)</code>	<code>bond_ytm(114.7202, 100, 0.08, 10, 1)</code>	0.0600000
<code>bond_duration(face, crate, y, n, freq)</code>	<code>bond_duration(100, 0.08, 0.06, 10, 1)</code>	7.44502
<code>bond_mduration(face, crate, y, n, freq)</code>	<code>bond_mduration(100, 0.08, 0.06, 10, 1)</code>	7.02360
<code>bond_convexity(face, crate, y, n, freq)</code>	<code>bond_convexity(100, 0.08, 0.06, 10, 1)</code>	65.1716
<code>amort(principal, i, n)</code>	<code>amort(1000, 0.01, 3).balance[3]</code>	4.26326e-12
<code>datenum(y, m, d)</code>	<code>datenum(2026, 7, 17)</code>	2.46124e+06
<code>datestr(jdn)</code>	<code>datestr(datenum(2026, 7, 17) + 90)</code>	"2026-10-15"

Function	Worked example	Result
<code>daterec(jdn)</code>	<code>daterec(datenum(2026, 7, 17))</code>	<code>{y = 2026.00, m = 7.00000, d = 17.00000}</code>
<code>dateadd(y, m, d, k)</code>	<code>dateadd(2024, 12, 31, 1)</code>	<code>{y = 2025.00, m = 1.00000, d = 1.00000}</code>
<code>today()</code>	<code>today() == datenum(now().y, now().m, now().d)</code>	<code>true</code>
<code>dow(y, m, d)</code>	<code>dow(2026, 7, 17)</code>	<code>5.00000</code>
<code>days(y1,m1,d1, y2,m2,d2)</code>	<code>days(2026, 1, 1, 2026, 7, 17)</code>	<code>197.000</code>
<code>days360(y1,m1,d1, y2,m2,d2)</code>	<code>days360(2024, 1, 31, 2024, 7, 31)</code>	<code>180</code>

4. astro.nu — the solar almanac

Sunrise, sunset, three grades of twilight, solar noon, day length, sun position, and moon phase, for any latitude and longitude (degrees; north and east positive). Times are local decimal hours given a UTC offset `tz`; `hm` renders them as “HH:MM”. NOAA solar position algorithm, within about a minute of the astral reference library (`tests/39_astro.test`). A `places` record preloads coordinates for several cities.

```
neutrino> load("packages/astro.nu")
neutrino> hm(sunrise(places.alexandria_va.lat, places.alexandria_va.lon, 2026, 7, 10, -4))
"05:52"
neutrino> hm(sunset(places.duluth_ga.lat, places.duluth_ga.lon, 2026, 7, 10, -4))
"20:50"
neutrino> hm(solar_noon(places.ljubljana.lon, 2026, 7, 10, 2))
"13:07"
neutrino> day_length(places.ljubljana.lat, places.ljubljana.lon, 2026, 7, 10)
15.5331
```

Planning a drive to arrive before dark — civil dawn at the origin to civil dusk at the destination:

```
neutrino> load("packages/astro.nu")
neutrino> let w = drive_daylight(places.alexandria_va, places.duluth_ga, 2026, 7, 10, -4)
{depart_dawn = 5.34382, depart_rise = 5.86728, arrive_set = 20.8376, arrive_dusk = 21.3161, window_h
neutrino> hm(w.depart_dawn) + " to " + hm(w.arrive_dusk)
"05:21 to 21:19"
neutrino> w.window_hours
15.9723
```

Sun position, moon illumination, and the honest polar refusal:

```
neutrino> load("packages/astro.nu")
error: the sun does not cross 90.833 degrees at latitude 80 on 2026-6-21
neutrino> let p = sun_position(38.8048, -77.0469, 2026, 7, 10, 13.22, -4)
{alt = 73.3594, az = 179.65}
neutrino> p.alt
73.3594
neutrino> moon_illum(2026, 7, 10)
0.19449
```

```
neutrino> sunrise(80, 0, 2026, 6, 21, 0)
```

Function	Worked example	Result
sunrise(lat, lon, y, m, d, tz)	hm(sunrise(38.8048, -77.0469, 2026, 7, 17, -4))	"05:57"
sunset(lat, lon, y, m, d, tz)	hm(sunset(38.8048, -77.0469, 2026, 7, 17, -4))	"20:31"
dawn_civil / dusk_civil(lat, lon, y, m, d, tz)	hm(dawn_civil(38.8048, -77.0469, 2026, 7, 17, -4))	"05:26"
dawn_nautical / dusk_nautical(lat, lon, y, m, d, tz)	hm(dawn_nautical(38.8048, -77.0469, 2026, 7, 17, -4))	"04:48"
dawn_astro / dusk_astro(lat, lon, y, m, d, tz)	hm(dusk_astro(38.8048, -77.0469, 2026, 7, 17, -4))	"22:23"
solar_noon(lon, y, m, d, tz)	hm(solar_noon(-77.0469, 2026, 7, 17, -4))	"13:14"
day_length(lat, lon, y, m, d)	day_length(46.0569, 14.5058, 2026, 7, 17)	15.3443
sun_position(lat, lon, y, m, d, hour, tz)	sun_position(38.8048, -77.0469, 2026, 7, 17, 13.2, -4).alt	72.2914
moon_age(y, m, d)	moon_age(2026, 7, 17)	2.70686
moon_illum(y, m, d)	moon_illum(2026, 7, 17)	0.0806581
hm(h)	hm(6.05)	"06:03"
places	places.duluth_ga.lat	34.0029
drive_daylight(from, to, y, m, d, tz)	drive_daylight(places.alexandria_la, places.duluth_ga, 2026, 7, 17, -4).window_hours	15.8284

5. rmt.nu — random matrices, structured

packages/rmt.nu is sugar over randn/qr/eye for the matrices you actually reach for at the prompt: symmetric, positive definite (chol-safe by construction: eigenvalues at least 1), Haar orthogonal, permutations and their matrices, correlation, row-stochastic (random Markov chains), and the Gaussian orthogonal ensemble — whose spectrum follows Wigner's semicircle on $[-2, 2]$, as the last line demonstrates on a 200×200 draw. Everything is reproducible under `rng(seed)`.

```
neutrino> load("packages/rmt.nu")
neutrino> format(6)
neutrino> rng(11)
neutrino> randorth(2)
[0.684752, 0.728776; -0.728776, 0.684752]
neutrino> let P = randspd(3); chol(P)[1, 1] > 0
true
neutrino> randperm(6)
[4, 5, 1, 2, 3, 6]
neutrino> permuat([3, 1, 2])
```

```
[0, 0, 1; 1, 0, 0; 0, 1, 0]
neutrino> let H = goe(200); max(abs(eig(H).values))
1.99476
```

Function	Worked example	Result
randsym(n)	let S = randsym(3); max(max(abs(S - S'))) == 0	true
randspd(n)	let P = randspd(3); min(eig(P).values) >= 1	true
wishart(n)	let W = wishart(3); min(eig(W).values) > 0	true
randorth(n)	let Q = randorth(3); max(max(abs(Q' * Q - eye(3)))) < 1e-12	true
randperm(n)	sort(randperm(5)) == 1:5	[true, true, true, true, true]
permmat(p)	permmat([2, 3, 1])	[0, 1, 0; 0, 0, 1; 1, 0, 0]
randcorr(n)	let C = randcorr(3); max(abs(diag(C) - 1)) < 1e-12	true
randstoch(n)	let T = randstoch(4); max(abs(sum(T, 2) - 1)) < 1e-12	true
goe(n)	let H = goe(150); max(abs(eig(H).values)) < 2.4	true

Writing your own

The pattern all four follow: a file of let definitions, assert-validated inputs, a record of closures when a namespace helps, and a golden test file whose reference values come from an independent implementation. Multi-line definitions are fine wherever a bracket is open. save your workspace, load it tomorrow — a package is just a workspace you chose to keep.